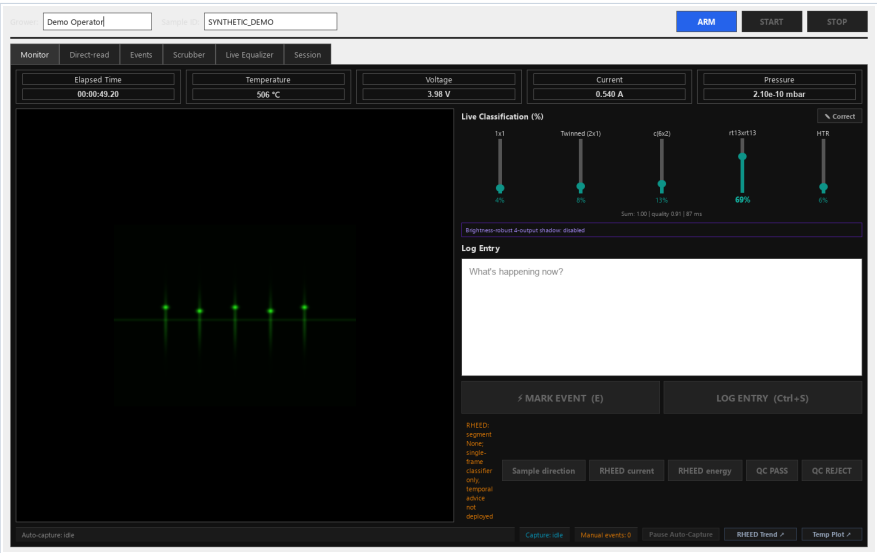


Ch-MBE GUI and RHEED

Post-processing Labeling

English Operator Manual



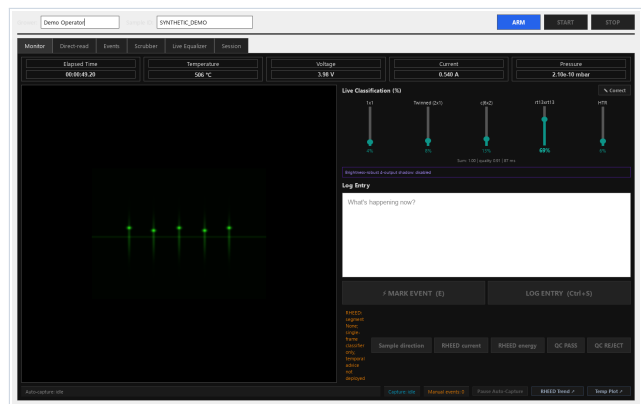
Actual application screenshot. The current Ch-MBE Growth Monitor is running without hardware; all visible values and the RHEED frame are generated demo inputs.

The post-processing labeler reads archived data only. It does not connect to or control cameras, pyrometers, power supplies, or any other instrument.

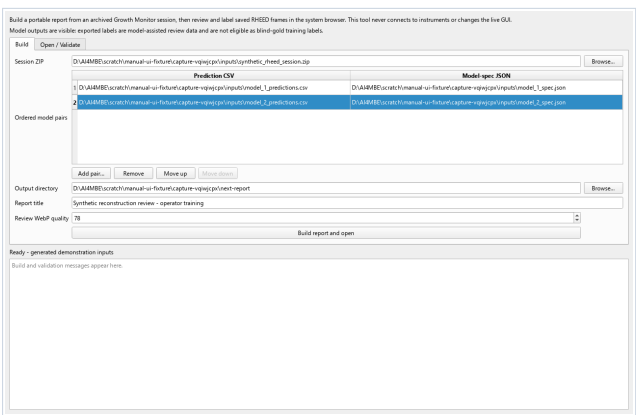
Quick entry	Double-click
Ch-MBE live GUI	Start Ch-MBE Growth Monitor.cmd
RHEED offline labeler	Start RHEED Post-processing Labeler.cmd
Install or refresh desktop shortcuts	Install AI4MBE Desktop Shortcuts.cmd

1 Understand the two independent paths

The live GUI performs acquisition and display. The offline labeler works only on completed archived sessions. Never interpret an offline result as a real-time control instruction.



1. Live acquisition
Growth Monitor writes the archived session.



2. Offline build and validation
The labeler combines the ZIP with predictions and model specifications.

The live GUI creates session records. The labeler reads those records later, produces a browser report, and exports validated JSON. It never sends commands back to the instruments.

What this manual covers

Page	Topic	Outcome
3	Shortcuts and live GUI startup	Confirm the correct chamber and a clean exit
4	Labeler desktop application	Select inputs, build, open, and validate
5-6	Inputs and safe report build	Produce a complete local report directory
7-8	Segment labeling and display controls	Create traceable labels on actual saved frames
9	Export, import, and validation	Produce canonical JSON and validate fail-closed
10-12	Safety, troubleshooting, and version checks	Handoff and reproduce the work safely

Surface-reconstruction labels, acquisition-quality QC, and FeSe film quality are three different concepts. The current classifier concerns the bare STO surface before growth.

Operating principle

- The GUI reads live instrument sources and writes session records under the operator's control.
- Post-processing combines an archived session with one or more already-generated prediction tables.
- Human labels are stored as frame-bound temporal segments with provenance.
- Model outputs remain visible, so these annotations are assisted review, not blind-gold labels.

2 Install shortcuts and start the Ch-MBE GUI

First setup or after moving the checkout

Step 1. From the GUI repository root, double-click `Install AI4MBE Desktop Shortcuts.cmd`.

Step 2. Confirm that the desktop contains refreshed shortcuts named **Ch-MBE Growth Monitor** and **RHEED Post-processing Labeler**.

Step 3. If Windows displays an unexpected security warning, do not bypass it. Stop and ask the maintainer to verify the file source and commit.

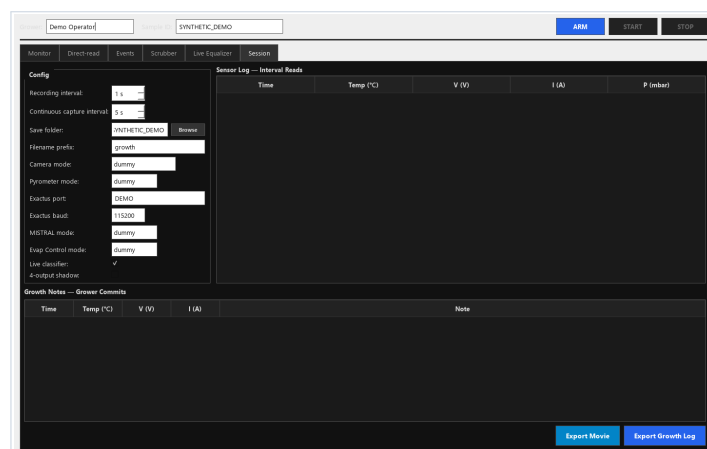
Start each experiment

Step 1. Double-click `Start Ch-MBE Growth Monitor.cmd` or its desktop shortcut. Do not use an O-MBE launcher.

Step 2. Wait for the main window and confirm the title **Chalcogenide MBE Growth Monitor**. Then inspect each device status.

Step 3. ARM or start a session only when the operator is ready under the experiment SOP. Double-click startup does not authorize setpoint changes.

Step 4. At the end, close through the GUI normally and wait for the window to disappear. Do not force-kill Python while logs are being written.



Actual application screenshot. The current Session tab in a hardware-free demo. Confirm the chamber and every selected interface before ARM.

The launcher forces the Ch-MBE chamber configuration and refuses a second live-GUI process through a Windows single-instance mutex.

Before ARM, confirm the Ch-MBE chamber, an advancing RHEED frame sequence, a valid temperature state, and the intended log directory.

Launcher logs are stored below `%LOCALAPPDATA%\AI4MBE\LauncherLogs`. They record the resolved Python interpreter, repository path, branch, commit, chamber, and optional driver status. Missing `pyads`, `serial`, or `windows_capture` produces a visible warning without hiding the GUI; do not ARM a production mode that needs the missing driver. Do not assume a fixed drive letter or create global environment variables ad hoc.

3 Open the RHEED post-processing labeler

Step 1. Double-click Start RHEED Post-processing Labeler.cmd or its desktop shortcut.

Step 2. Confirm that the application has the **Build** and **Open / Validate** tabs, plus **Build report and open**, **Open report**, **Validate annotations**, and **Open English PDF manual**. It must not start Growth Monitor or contact instruments.

Build a portable report from an archived Growth Monitor session, then review and label saved RHEED frames in the system browser. This tool never connects to instruments or changes the live GUI. Model outputs are visible: exported labels are model-assisted review data and are not eligible as blind-gold training labels.

Build Open / Validate

Session ZIP D:\AI4MBE\scratch\manual-ui-fixture\capture-vqiwjcp\inputs\synthetic_rheed_session.zip Browse...

	Prediction CSV	Model-spec JSON
1	D:\AI4MBE\scratch\manual-ui-fixture\capture-vqiwjcp\inputs\model_1_predictions.csv	D:\AI4MBE\scratch\manual-ui-fixture\capture-vqiwjcp\inputs\model_1_spec.json
2	D:\AI4MBE\scratch\manual-ui-fixture\capture-vqiwjcp\inputs\model_2_predictions.csv	D:\AI4MBE\scratch\manual-ui-fixture\capture-vqiwjcp\inputs\model_2_spec.json

Ordered model pairs

Add pair... Remove Move up Move down

Output directory D:\AI4MBE\scratch\manual-ui-fixture\capture-vqiwjcp\next-report Browse...

Report title Synthetic reconstruction review - operator training

Review WebP quality 78

Build report and open

Ready - generated demonstration inputs

Build and validation messages appear here.

Actual application screenshot. The current desktop labeler Build tab populated with a generated local fixture. The application is offline and no instrument process is running.

Area	Purpose
Session ZIP	Select one archived Growth Monitor session
Model inputs	Pair one prediction CSV with its model-spec JSON on each row
Output directory	Select a new or empty directory, preferably outside Git
Message log	Preserve complete build and validation messages
Open report button	Open a previously generated interactive_report.html
Validate annotations	Check exported JSON against the exact report provenance

This application can run on a computer without instrument software when the archived inputs are complete and the Python environment is valid. Report construction and validation run in an isolated child process so the desktop application remains responsive.

4 Prepare inputs and verify provenance

Session archive

The ZIP archive must contain exactly one member ending in `session_metadata.json`, exactly one member ending in `heartbeat_log.csv`, and every sibling frames/ image referenced by the heartbeat rows. Saved-frame elapsed time, heartbeat index, capture sequence, and timezone-aware capture UTC must each increase strictly; gaps are allowed. Do not extract and rename frames manually.

```
growth_session.zip
  session_metadata.json
  heartbeat_log.csv
  frames/
    heartbeat_000001_....bmp
    heartbeat_000002_....bmp
```

Prediction tables and model specifications are positional pairs

File	Required content	Check
Prediction CSV	frame index, heartbeat index, elapsed time, capture UTC, capture sequence, frame name, frame SHA-256, and score columns	Rows match the saved-frame order exactly
Model-spec JSON	schema version 1, non-empty key and title, at least two unique classes, and equally sized unique probability columns	Class order matches the score columns; status and provenance are optional
Session ZIP	metadata, heartbeat log, and frames	Every referenced frame exists in the archive

Prediction `frame_index` may begin at 0 or 1, but it must then remain contiguous. Heartbeat indices, capture sequences, and elapsed times may contain gaps. The tool follows actual saved frames and recorded times; it never assumes exact 1 Hz sampling.

Any mismatch in row count, order, time, sequence, filename, or SHA-256 stops the build. Do not edit the inputs to bypass an error.

Recommended local layout

```
<local-data-root>\2026-08-11_anneal\
  source\growth_session.zip
  predictions\model_A.csv
  specs\model_A.json
  output\
  exports\
```

5 Build and open a report safely

Step 1. Choose the preserved archive copy in Session ZIP.

Step 2. Add one Prediction CSV and Model-spec JSON row for every model to display. Pairing is positional, so preserve row order.

Step 3. Choose a new or empty directory outside the repository. The desktop labeler will not overwrite a non-empty directory.

Step 4. Enter a report title. Review quality changes only the lossy review images; it never changes archived pixels or model predictions. The default value of 78 is suitable for routine use.

Step 5. Click **Build report and open**. Wait for success and for `interactive_report.html` to open automatically.

PowerShell fallback

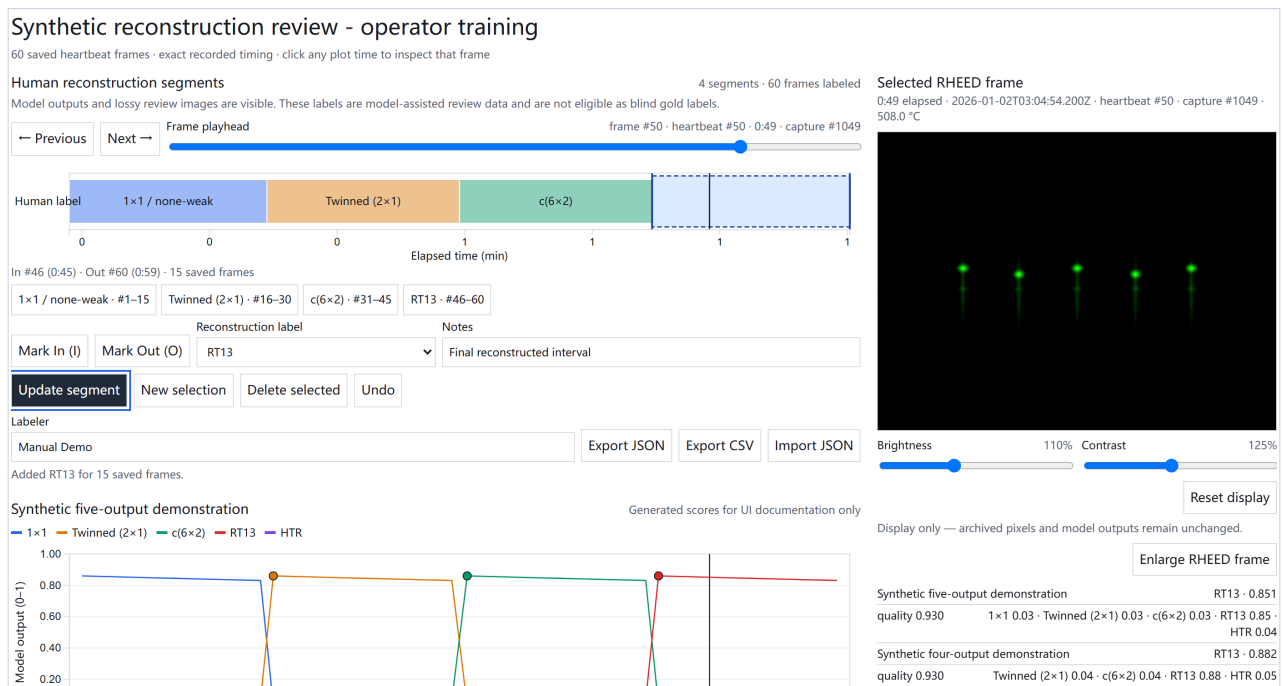
```
Set-Location '<GUI repository>'
conda activate ai4mbe-gui
python -m tools.rheed_postprocessing_labeling build `
  --session '<local-data-root>\growth_session.zip' `
  --predictions '<local-data-root>\predictions.csv' `
  --model-spec '<local-data-root>\model_spec.json' `
  --output-dir '<local-data-root>\labeling-output'
```

Copy the entire report directory

The report consists of `interactive_report.html`, `images/`, `vendor/`, and `run_manifest.json`. Copy the entire directory during handoff. Preserve the original prediction CSV and model-spec JSON separately outside the report directory.

The desktop application always refuses a non-empty output directory. CLI `--overwrite` may replace only a recognized, unmodified tool output. It validates the new inputs and stages a complete replacement first. If the old report changes during the build, replacement stops and the original content is restored. Filesystem roots, repository paths, the current directory, and directories containing inputs are always rejected.

6 Label temporal segments like an editing timeline



Actual application screenshot. The current English browser editor with generated RHEED frames, synthetic model curves, saved human segments, and a synchronized playhead.

Step 1. Move the Frame playhead or click a model plot to locate the first saved frame of a segment. Click **Mark In**.

Step 2. Move to the final saved frame of the segment and click **Mark Out**. The Out marker is inclusive.

Step 3. Choose a reconstruction label, enter the labeler name, add notes when useful, and click **Add segment**.

Step 4. Select a saved segment to edit it. Update preserves its annotation ID. Adjacent segments are allowed; overlapping segments are rejected.

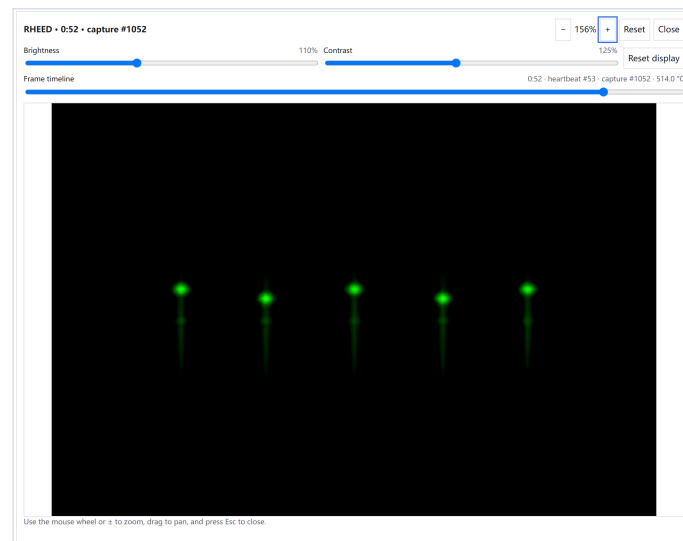
The displayed and exported saved-frame ordinal is 1-based. Every endpoint also records heartbeat index, capture sequence, UTC, and frame SHA-256. This preserves traceability even when the sampling interval or sequence contains gaps.

Labels cover the saved frames from In through Out, inclusive. They do not claim that unsaved camera frames or every capture-sequence number in between was reviewed.

7 Review, display controls, and drafts

All time indicators remain synchronized

Action	Must update together	Must not change
Move the main timeline	Current RHEED image, model guides, enlarged-view timeline	Archive and prediction values
Click a model curve	Main playhead, image, and metadata	Saved segments
Adjust brightness or contrast	On-screen appearance	Pixels, SHA-256, and model outputs
Enlarge the image	Zoom view and its adjustable timeline	Source image and segment boundaries



Actual application screenshot. The actual enlarged-frame dialog. Its timeline changes the selected frame while preserving zoom, pan, and display adjustments.

Drafts and editing

- The browser stores a draft scoped by both dataset ID and model-context fingerprint in localStorage. This is not a durable backup.
- Export JSON frequently, especially during long reviews and before changing computers, browsers, or cache settings.
- Delete selected requires confirmation. Undo restores only the most recent annotation mutation.
- Import JSON validates before replacement. On failure, the current segment set remains unchanged.

Interpretation

Labels describe the dominant surface-reconstruction category seen by the reviewer during the interval. The displayed **Uncertain** label exports as unknown; **1x1 / none-weak** exports as none_weak. Never overwrite a human selection with model argmax, and never interpret unknown as acquisition-quality rejection.

Brightness, contrast, zoom, and image enlargement aid inspection only. Brightness and contrast are stored with each segment; zoom is not exported. The report never rewrites archived images or model outputs.

Use a four-pass review rhythm: survey the full run, confirm every In and Out frame, inspect gaps and uncertain labels, then export and validate the canonical JSON.

8 Export, import, and fail-closed validation

JSON is the canonical artifact

Use Export JSON to resume work, validate provenance, and support downstream processing. Export CSV is useful for meetings and tabular analysis, but it does not replace the complete nested provenance in JSON.

Binding	Purpose
dataset_id and source archive SHA-256	Bind the source session
Ordered-frame fingerprint	Bind frame order and per-frame SHA-256
model_context_fingerprint	Bind the visible predictions and model specifications
Endpoint provenance	Bind saved-frame ordinal, heartbeat, UTC, sequence, and SHA-256
model_outputs_visible=true	Disclose that the models were visible
eligible_for_gold=false	Prevent blind-gold misuse

Validate in the desktop application

Step 1. Select the original `interactive_report.html`.

Step 2. Select the exported annotation JSON.

Step 3. Click **Validate annotations**. Continue only when the application says **Annotation JSON is valid for this report**.

PowerShell validation

```
python -m tools.rheed_postprocessing_labeling validate `
  --report '<local-data-root>\labeling-output\interactive_report.html' `
  --annotations '<local-data-root>\exports\segment_annotations.json'
```

A wrong run, changed model context, altered endpoint, overlapping segment, invalid label, or gold-data claim causes validation to fail. Return to the correct report and original export and investigate the exact message.

Browser Import performs immediate client-side provenance checks before replacing the current draft. Desktop **Validate annotations** is the authoritative full fail-closed validation.

9 Data safety and scientific boundaries

What belongs in Git

May be tracked	Keep out by default
Tool code, templates, tests, and documentation	Real session ZIP files, raw RHEED images, and sensor logs
Example model specifications	Real predictions, generated reports, and annotation JSON or CSV
Sanitized documentation screenshots made with generated demo inputs	Checkpoints, unpublished results, and experiment-specific screenshots

Safe handling sequence

- Step 1. Keep the source session ZIP read-only or preserve an immutable copy, and record its SHA-256.
- Step 2. Create a dedicated working directory outside the repository. Keep reports, exports, and screenshots there.
- Step 3. For handoff, copy the complete report directory and canonical JSON. Separately preserve the original prediction CSV, model-spec JSON, and SHA manifest. Never send only the HTML.
- Step 4. Run statistics or training-data preparation only after validation, and preserve the original export unchanged.

Model plots and lossy review images are visible during labeling. These annotations are eligible only as model-assisted review. Formal blind-gold labels require a separate workflow that hides model and Equalizer outputs and preserves the required audit evidence.

The current classification target is bare STO surface reconstruction before growth. Do not interpret a reconstruction label as FeSe film quality. Do not equate reconstruction unknown with acquisition-quality QC_REJECT.

10 Troubleshooting

First preserve the exact error text, occurrence time, branch, commit, and selected paths. Do not begin by deleting environments, changing global variables, or force-resetting Git. A live-driver warning is fail-visible behavior, not a prompt for ad hoc dependency installation.

Symptom	Likely cause	Safe action
Desktop shortcut is absent or opens an old checkout	Installer was not run after the check-out moved	Run the shortcut installer from the current repository root
Growth Monitor shows the wrong chamber	Wrong launcher was used	Close normally and use only the Ch-MBE launcher
Launcher window closes immediately	Environment or dependency failure	Read the newest launcher log; if needed, run the same CMD from PowerShell
Temperature or RHEED has no valid reading	Interface, vendor window, or selected mode issue	Record connected, error, and mode; do not guess global settings
Labeler rejects Build	Missing input, mismatched pair, or non-empty output	Check the session, each positional pair, and a new output directory
Provenance mismatch	Predictions do not belong to the ZIP or rows changed	Find the matching predictions and spec; do not edit the CSV
HTML has no images or curves	Only the HTML was copied or assets are missing	Restore the complete report directory and relative paths
Draft disappeared	Browser, computer, or localStorage changed	Import the latest JSON and export more frequently
Import or validation fails	Run, context, endpoint, overlap, or label mismatch	Use the original report and JSON; read the fail-closed message

Minimum evidence for the maintainer

- Full error text and occurrence time. Do not report only that it does not work.
- Output of `git status --short --branch` and `git rev-parse HEAD`.
- Launcher name, environment path, input filenames, and output directory. Keep sensitive data out of public channels.
- For labeling issues, report-manifest and annotation-JSON SHA values. Raw images are not initially required.

11 Version verification and quick checklist

Record versions before acquisition or labeling

```
Set-Location '<GUI repository>'
git status --short --branch
git rev-parse HEAD
conda env list
conda activate ai4mbe-gui
python --version
python -m tools.rheed_postprocessing_labeling --help
```

Use the resolved Python interpreter and repository path recorded by the launcher. Read AI4MBE_GUI_PYTHON only when it is already configured on that workstation; do not create it ad hoc. If the branch or commit differs from the team-specified version, or Git reports unknown modifications, stop and verify.

Integrity hashes

```
Get-FileHash '<local-data-root>\growth_session.zip' -Algorithm SHA256
Get-FileHash '<local-data-root>\exports\segment_annotations.json' -Algorithm SHA256
Get-FileHash '.\docs\RHEED_GUI_Postprocessing_Labeling_User_Manual_EN.pdf' -Algorithm SHA256
```

Quick checklist

- ☐ Use the Ch-MBE launcher and confirm the chamber identity.
- ☐ Confirm live status, frame sequence, temperature, and logging behavior under the experiment SOP.
- ☐ Preserve a read-only source ZIP and its SHA-256.
- ☐ Open the offline labeler with the dedicated launcher.
- ☐ Pair every prediction CSV with the correct model-spec JSON.
- ☐ Use a new or empty output directory outside Git.
- ☐ Review boundaries frame by frame; allow no overlaps.
- ☐ Export JSON and validate it against the original report.
- ☐ Record branch, commit, environment, manifest, and export hashes.
- ☐ Handoff the complete report and declare model-assisted, not blind-gold.

Stop and contact the maintainer when instrument state, stale data, version identity, provenance, or validation cannot be explained.